

Lecture-#7

Natural Language Processing

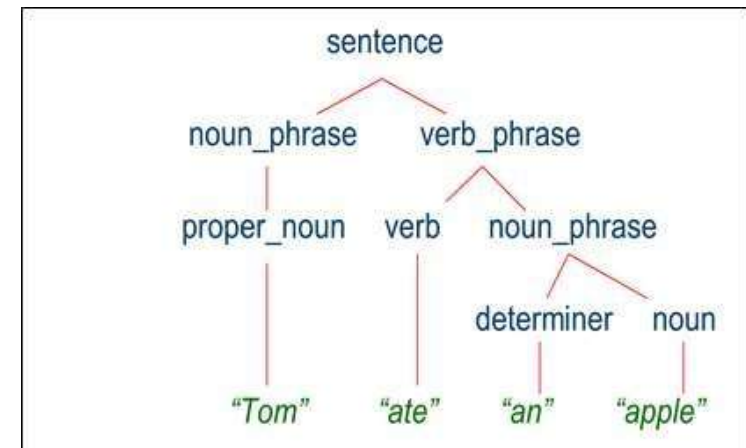
A8706

Syntax Analysis

Parsing Natural Language

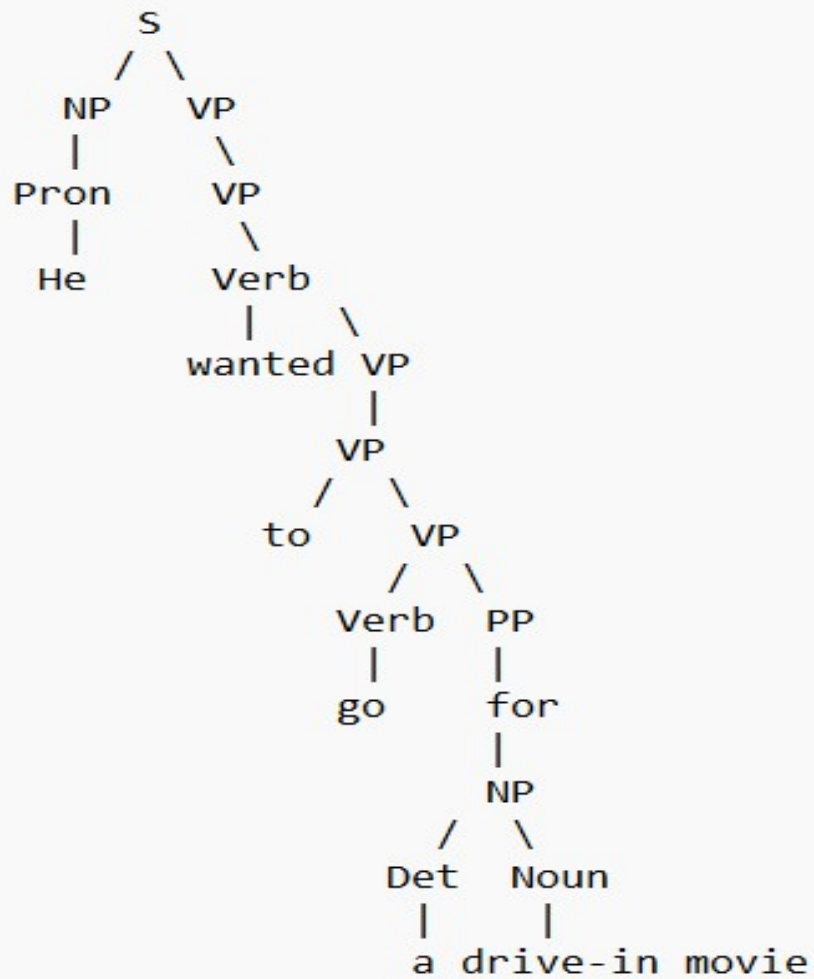
The parsing in NLP is the process of determining the syntactic structure of a text by analysing its constituent words based on an underlying grammar.

```
sentence -> noun_phrase, verb_phrase  
noun_phrase -> proper_noun  
noun_phrase -> determiner, noun  
verb_phrase -> verb, noun_phrase  
proper_noun -> [Tom]  
noun -> [apple]  
verb -> [ate]  
determiner -> [an]
```



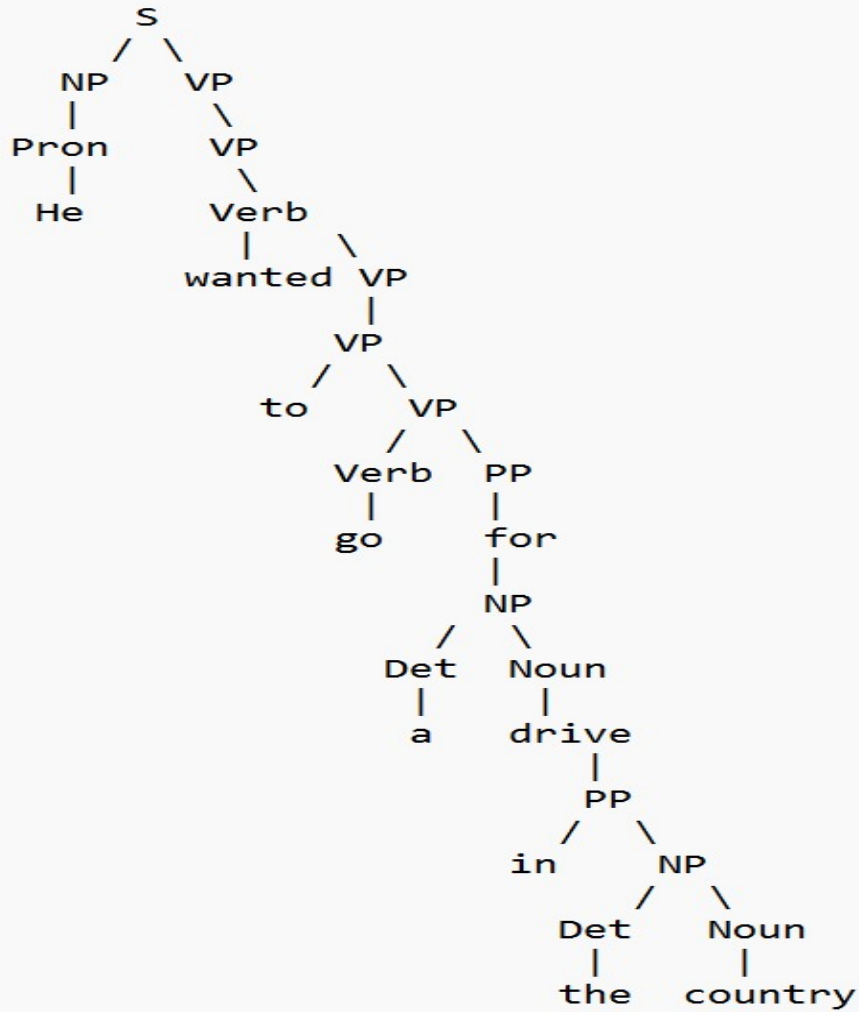
Then, the **outcome** of the parsing process would be a **parse tree**, where **sentence** is the root, intermediate nodes such as **noun_phrase**, **verb_phrase** etc. have children - hence they are called **non-terminals** and finally, the leaves of the tree '**Tom**', '**ate**', '**an**', '**apple**' are called **terminals**.

Parsing Natural Language



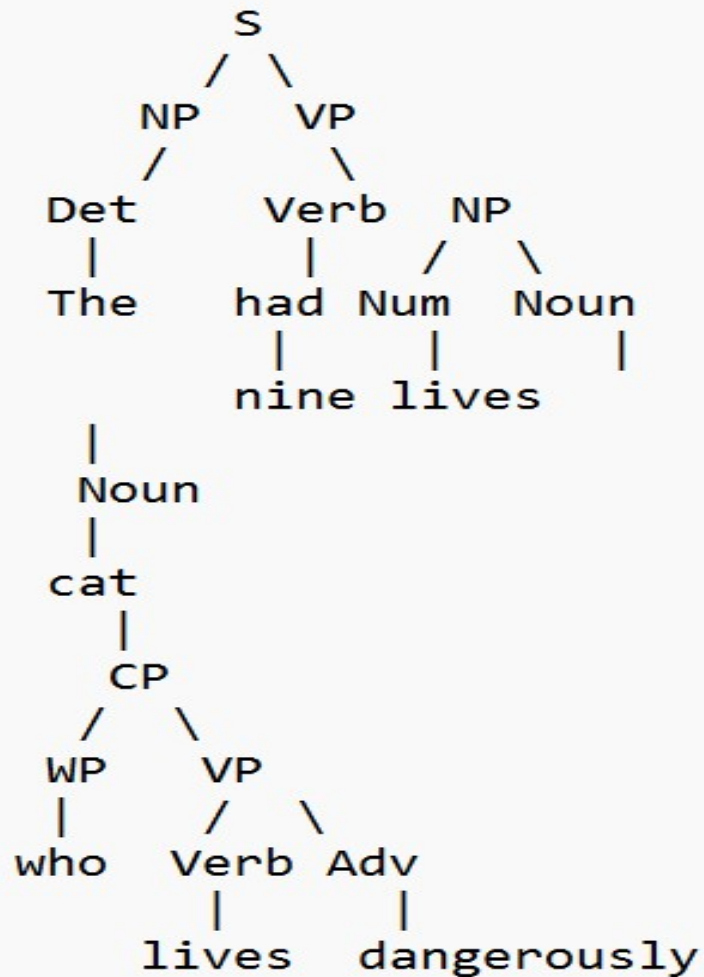
He wanted to go for a
drive in movie.

Parsing Natural Language



He wanted to go for a
drive in the country.

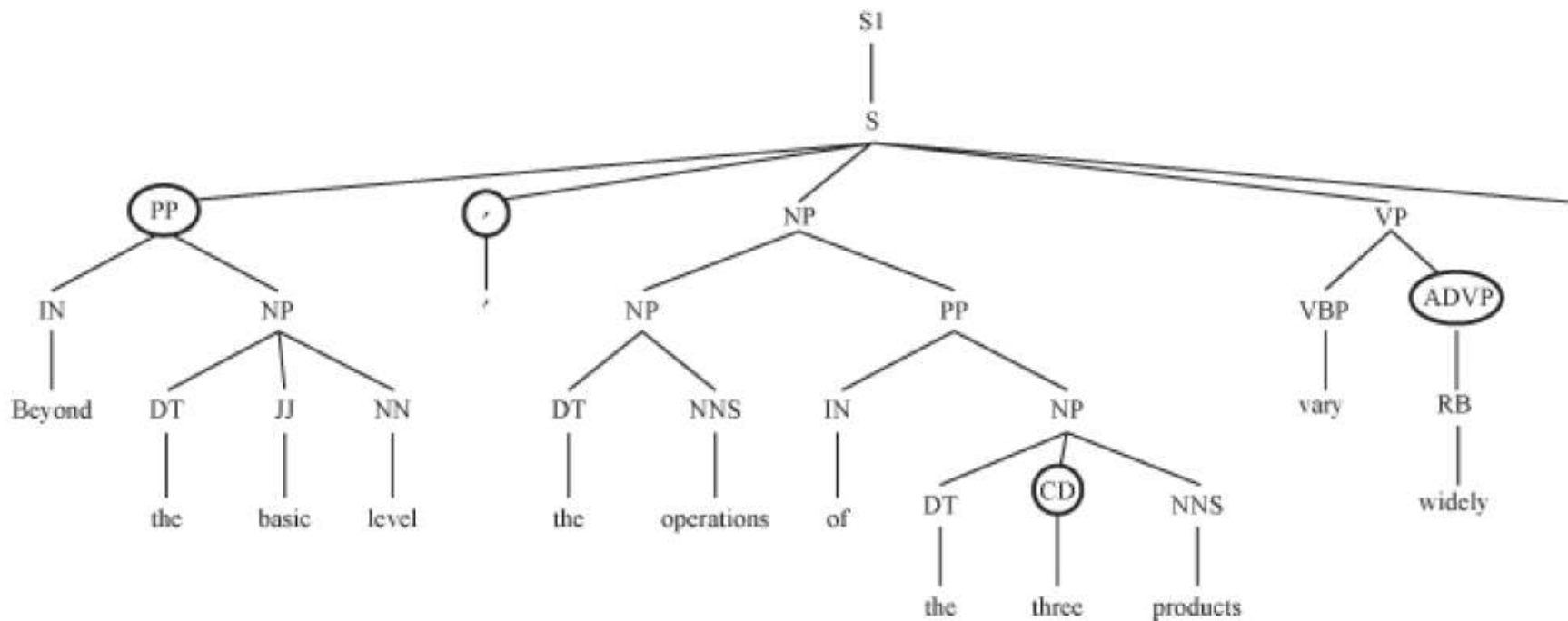
Parsing Natural Language



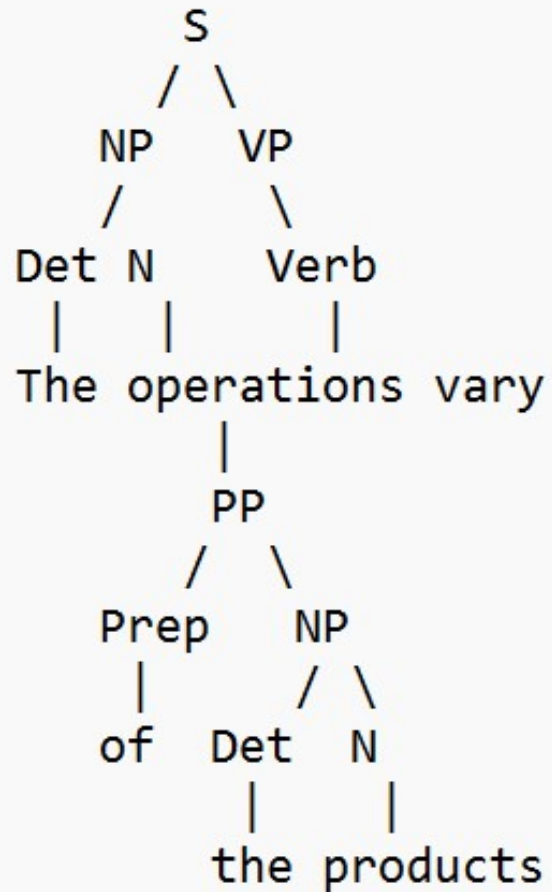
**The cat who lives
dangerously had nine lives.**

Parsing Natural Language

Beyond the basic level, the operations of the three products vary widely.



Parsing Natural Language



The operations of the products vary.

Parsing Natural Language



Learning from Experience, Sharing with Passion

- ❖ A treebank can be defined as a linguistically annotated corpus that includes some kind of syntactic analysis over and above part-of-speech tagging.
- ❖ A sentence is parsed by relating each word to other words in the sentence which depend on it.
- ❖ The syntactic parsing of a sentence consists of finding the correct syntactic structure of that sentence in the given formalism/grammar.
- ❖ Dependency grammar (DG) and phrase structure grammar (PSG) are two such formalisms.
- ❖ PSG breaks sentence into constituents (phrases), which are then broken into smaller constituents.
- ❖ Describe phrase, clause structure Example: NP, PP, VP etc.,
- ❖ DG: syntactic structure consist of lexical items, linked by binary asymmetric relations called dependencies.
- ❖ Interested in grammatical relations between individual words.
- ❖ Does propose a recursive structure rather a network of relations
- ❖ These relations can also have labels.

Parsing Natural Language



Learning from Experience, Sharing with Passion

What Is a Treebank?

A treebank is a collection of sentences paired with detailed syntactic analyses—most often in the form of parse trees (constituent or dependency).

Why Use Treebanks?

- Provide **gold-standard** data for training and evaluating parsers
- Anchor and test **linguistic theories** of syntax
- Enable downstream NLP tasks (semantic role labeling, MT, etc.) to exploit syntax

Two Main Annotation Styles

1. Constituency (Phrase-Structure)

- Sentences broken into nested constituents (NP, VP, PP...)
- Commonly bracketed in Penn-Treebank style

2. Dependency

- Words linked by head–dependent relations
- Focuses on direct grammatical relationships (subject, object...)

Parsing Natural Language



Learning from Experience, Sharing with Passion

Key Treebank Projects

Penn Treebank (PTB)

- ~50,000 Wall Street Journal sentences
- Standard POS tags (NN, VBZ, DT...)
- Bracketed phrase-structure trees

Universal Dependencies (UD)

- Multilingual, harmonized POS tags and dependency labels
- Covers dozens of languages and genres

How a Treebank Is Built

- Text selection (newswire, fiction, web...)
- Automatic tokenization + POS-tagging
- Manual correction by trained annotators
- Tree annotation (constituent or dependency)
- Quality control and adjudication

Common Applications

- **Parser development:** train statistical or neural models
- **Information extraction:** improve relation and event extraction
- **Machine translation:** syntax-aware translation models
- **Linguistic research:** study syntactic patterns and constructions

Challenges & Limitations

- **High cost:** manual annotation is labor-intensive
- **Domain bias:** many treebanks focus on news text
- **Scheme variability:** different formats require conversion for combined use

Parsing Natural Language

```
(N (N (N natural)
      (N language))
 (N processing))
```

```
(N (N natural)
   (N (N language)
       (N processing)))
```

The Ambiguous Recursive Rule

- Suppose our grammar has (among others) this rule for stacking modifiers or nouns:
- $NP \rightarrow NP N$
and a base case like
- $NP \rightarrow \text{"book"} \mid \text{"processing"} \mid \text{"language"} \mid \text{"natural"}$
When you string together $k + 1$ base nouns ("natural language processing book"), each application of
- $NP \rightarrow NP N$
attaches one more terminal on the right, but leaves **where** you attach it ambiguous.

The Combinatorial Problem

- Parsing “ $w_0 w_1 \dots w_k$ ” under that single binary-branching rule is exactly the problem of **parenthesizing** (or building a full binary tree over) $k + 1$ leaves.
- Each parse corresponds to one way of inserting parentheses in a product of $k + 1$ factors:

$$w_0 \cdot w_1 \cdot \dots \cdot w_k$$

There is no crossing or reordering—just different ways to group them.

Catalan Numbers Count Binary Trees

The number of ways to fully parenthesize $n + 1$ items (or equivalently, the number of distinct full binary trees with n internal nodes) is the n -th **Catalan number** $\text{Cat}(n)$.

Closed-form:

$$\text{Cat}(n) = \frac{1}{n+1} \binom{2n}{n}$$

So for n recursions, you get $\text{Cat}(n)$ distinct parses.

Parsing Natural Language

k (recursions)	leaves = k+1	parses = C_k
0	1	$C_0 = 1$
1	2	$C_1 = 1$
2	3	$C_2 = 2$
3	4	$C_3 = 5$
4	5	$C_4 = 14$
5	6	$C_5 = 42$
6	7	$C_6 = 132$

“natural language processing book” uses $k = 3$ applications of $NP \rightarrow NP N \rightarrow$ **5** parses ($C_3=5$).

– Adding yet another word ($k=4$) jumps to 14 parses, and so on.

Parsing Natural Language

Compact Representation: Parse Forests

- ❑ Even though there may be exponentially many full trees, modern CFG parsers (e.g. CYK) build a **packed parse forest** in $O(n^3)$ time (worst case) and $O(n^2)$ space.
- ❑ This forest encodes all parses without enumerating them one-by-one.

The Second Bottleneck: Disambiguation

❑ Rule Acquisition

Which recursive rules (NP coordination, PP attachment, etc.) are allowed?

Preference Acquisition

Which of the many valid parses is most plausible for a given sentence?

Relies on lexical, semantic and contextual cues.

Treebanks as a Unified Solution

- ❑ A **treebank** is a large corpus of sentences manually annotated with their “correct” parse trees.
 - **Addresses both bottlenecks:**
- ❑ Explicitly records which grammatical constructions actually occur.
- ❑ Provides data to learn statistical or neural models that score/rank competing parses.
 - **Example:** Penn Treebank tags coordination and PP-attachments in Wall Street Journal text; a parser trained on it learns both the rules and their preferences.

Parsing Natural Language

- ❑ Parsers use the CFG (or dependency rules) to build a parse forest in polynomial time.
- ❑ A **statistical/neural disambiguation model**, trained on treebank data, then selects the single best parse (or k-best) for downstream tasks.
- ❑ In this way, treebanks overcome the **combinatorial explosion** of possible structures, and the **knowledge-acquisition** challenge of deciding which structure is correct in context.

Shallow Parsing

A parser takes input in the form of sequence of tokens and produces output in the form of parse tree.

•Key Elements in Shallow Parsing:

- **Noun Phrase (NP)**: A chunk that contains a noun and its modifiers.
 - Example: "**The red car**" in the sentence "The red car is fast."
- **Verb Phrase (VP)**: A chunk that contains a verb and any objects or modifiers.
 - Example: "**is driving fast**" in the sentence "He is driving fast."
- **Prepositional Phrase (PP)**: A chunk that starts with a preposition and includes its object.
 - Example: "**in the park**" in the sentence "She sat in the park."

Shallow Parsing

- Shallow parsing typically proceeds through these steps:
 - **Tokenization**: Breaking the text into individual words or tokens.
 - **POS Tagging**: Assigning part-of-speech tags (e.g., noun, verb, adjective) to each token.
 - **Chunking**: Grouping the tokens into larger units (phrases) based on syntactic patterns and rules.
- **Example of Shallow Parsing**:
 - For the sentence "**The cat sat on the mat,**" shallow parsing would break it down into chunks like this:
 - **[NP The cat] [VP sat] [PP on [NP the mat]]**

Shallow Parsing



Learning from Experience, Sharing with Passion

•Shallow Parsing vs. Full Parsing:

- **Shallow Parsing:** Focuses on identifying higher-level structures (phrases) without going deeper into hierarchical relations between words. It's faster and often used for information extraction, named entity recognition, and simple syntactic analysis.
- **Full Parsing (Deep Parsing):** Constructs a complete parse tree representing the full syntactic structure of the sentence, capturing relationships between all parts of the sentence. It's more computationally intensive and used for deeper linguistic analysis.

•Applications of Shallow Parsing:

- **Information Extraction:** Identifying key pieces of information (e.g., names, locations) in text.
- **Named Entity Recognition (NER):** Finding and classifying entities (e.g., people, organizations) within a sentence.
- **Text Summarization:** Simplifying text by extracting relevant chunks.

Shallow Parsing

Modeling a Sentence using Phase Structure

Every sentence consists of an internal structure which could be modeled with the phrase structure.

Algorithm : Steps

- ‡ Apply rules on an proposition
- ‡ The base proposition would be :
 S (the root, ie the sentence).
- ‡ The first production rule would be :
 (NP = noun phrase, VP = verb phrase)
 S -> (NP, VP)
- ‡ Apply rules for the 'branches'
 NP -> noun **VP -> verb, NP**
- ‡ The verb and noun have terminal nodes which could be any word in the lexicon for the appropriate category.
- ‡ The end is a tree with the words as terminal nodes, which is referred as the sentence.

Shallow Parsing

Statement: He ate the pizza

$G = (V, T, P, S)$

V- set of variables = { S, NP, VP, DT, N, V, PRO }

T- Set of terminals = { he, ate, the, Pizza }

P- Production Rule

S- start sym.

P: { $S \rightarrow NP \ VP$

$NP \rightarrow PRO$

$NP \rightarrow ART \ N$

$VP \rightarrow V \ NP$

$PRO \rightarrow He$

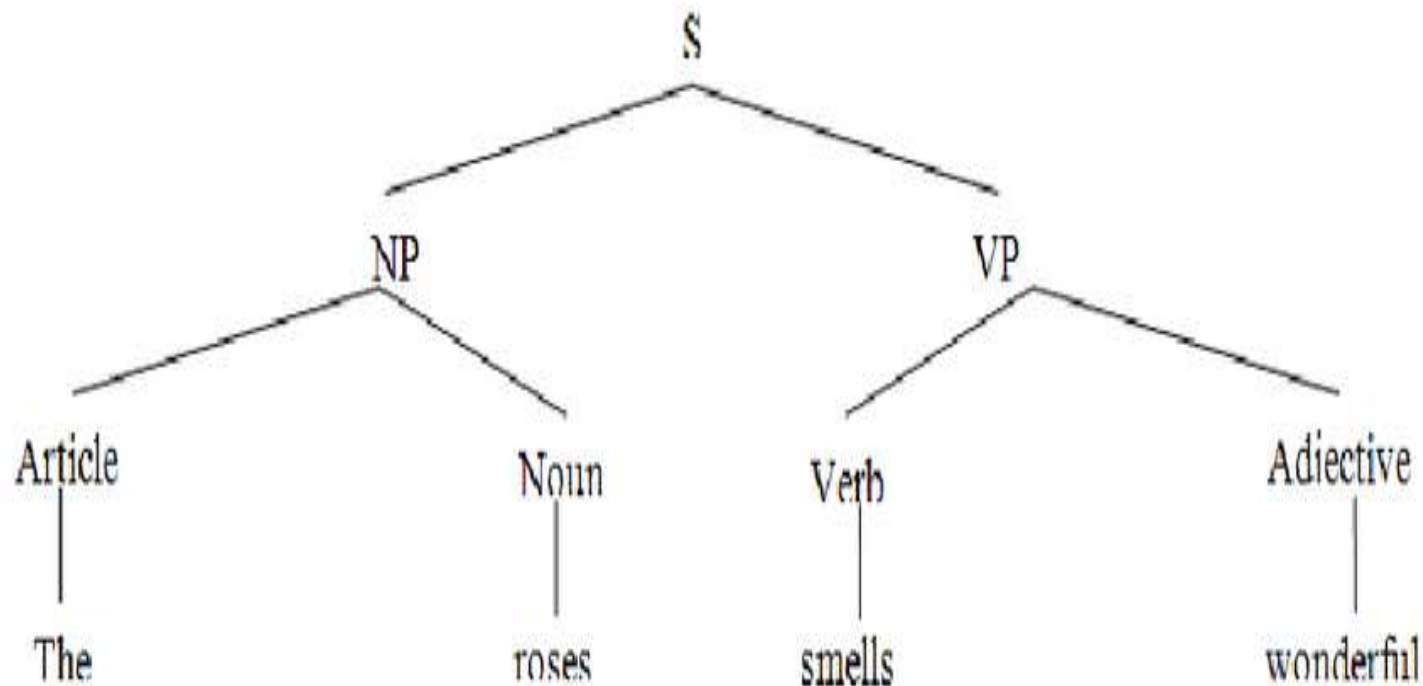
$DT \rightarrow the$

$V \rightarrow ate$

$N \rightarrow Pizza \}$

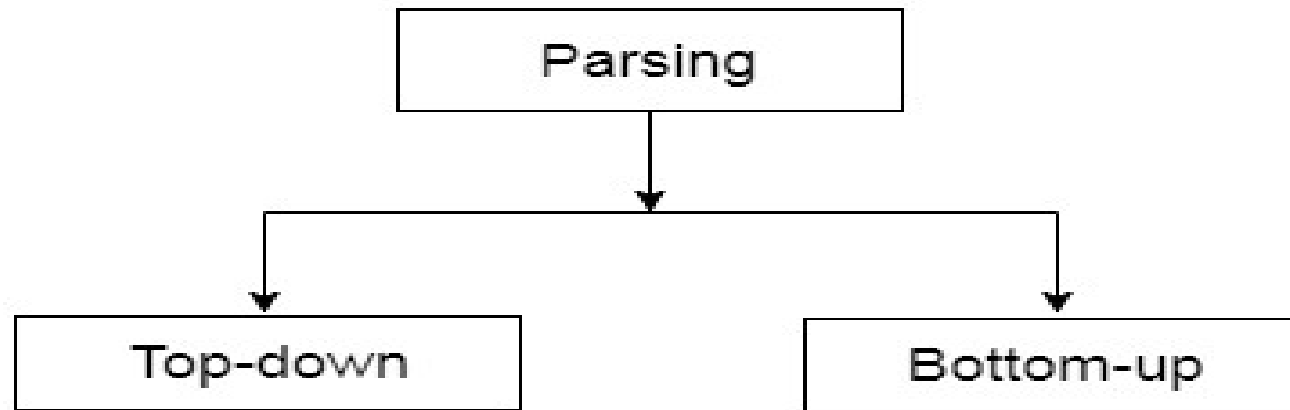
Shallow Parsing

Parse tree for the sentence “The rose smells wonderful”



Shallow Parsing

Parsing is of two types: top down parsing and bottom up parsing.

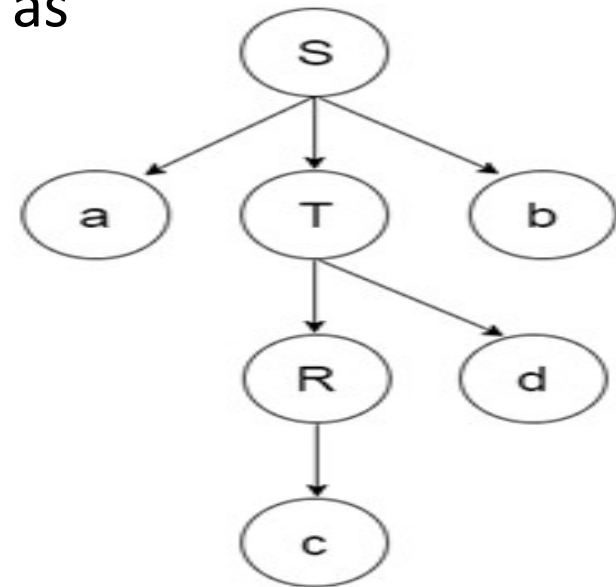


Shallow Parsing

Top down parsing

- ❖ The top down parsing is known as recursive parsing or predictive parsing.
- ❖ In the top down parsing, the parsing starts from the start symbol and transform it into the input symbol.

Parse Tree representation of input string "acdb" is as follows:



Shallow Parsing

Bottom up parsing

- ❖ Bottom up parsing is also known as shift-reduce parsing.
- ❖ Bottom up parsing is used to construct a parse tree for an input string.
- ❖ In the bottom up parsing, the parsing starts with the input symbol and construct the parse tree up to the start symbol by tracing out the rightmost derivations of string in reverse.

Parse Tree representation of input string "id * id" is as follows:

$E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow id$
 $F \rightarrow T$
 $F \rightarrow id$

id * id

Step 1

F * id
|
id

Step 2

T * id
|
F
|
id

Step 3

T * id
|
F
|
id

Step 4

T
/ | \
T * F
/ \
F id
|
id

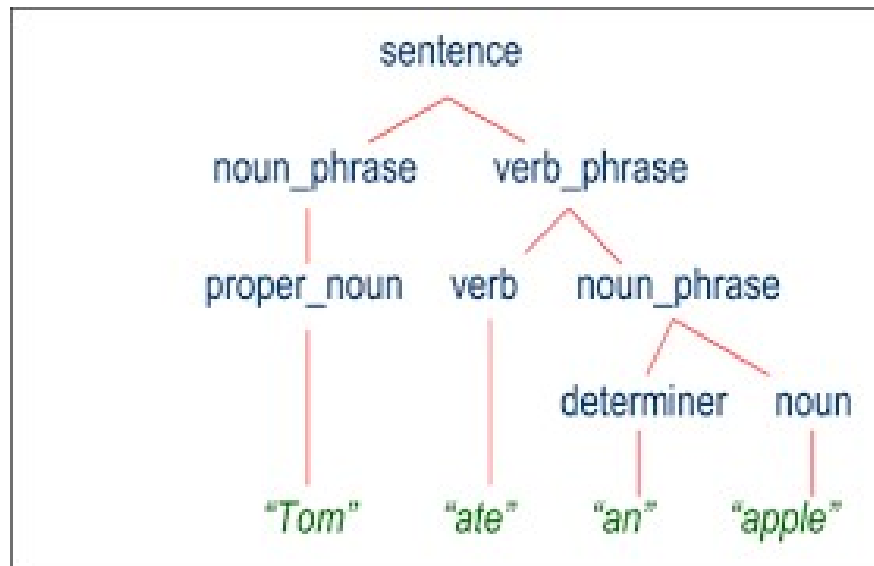
Step 5

E
|
T
/ | \
T * F
/ \
F id
|
id

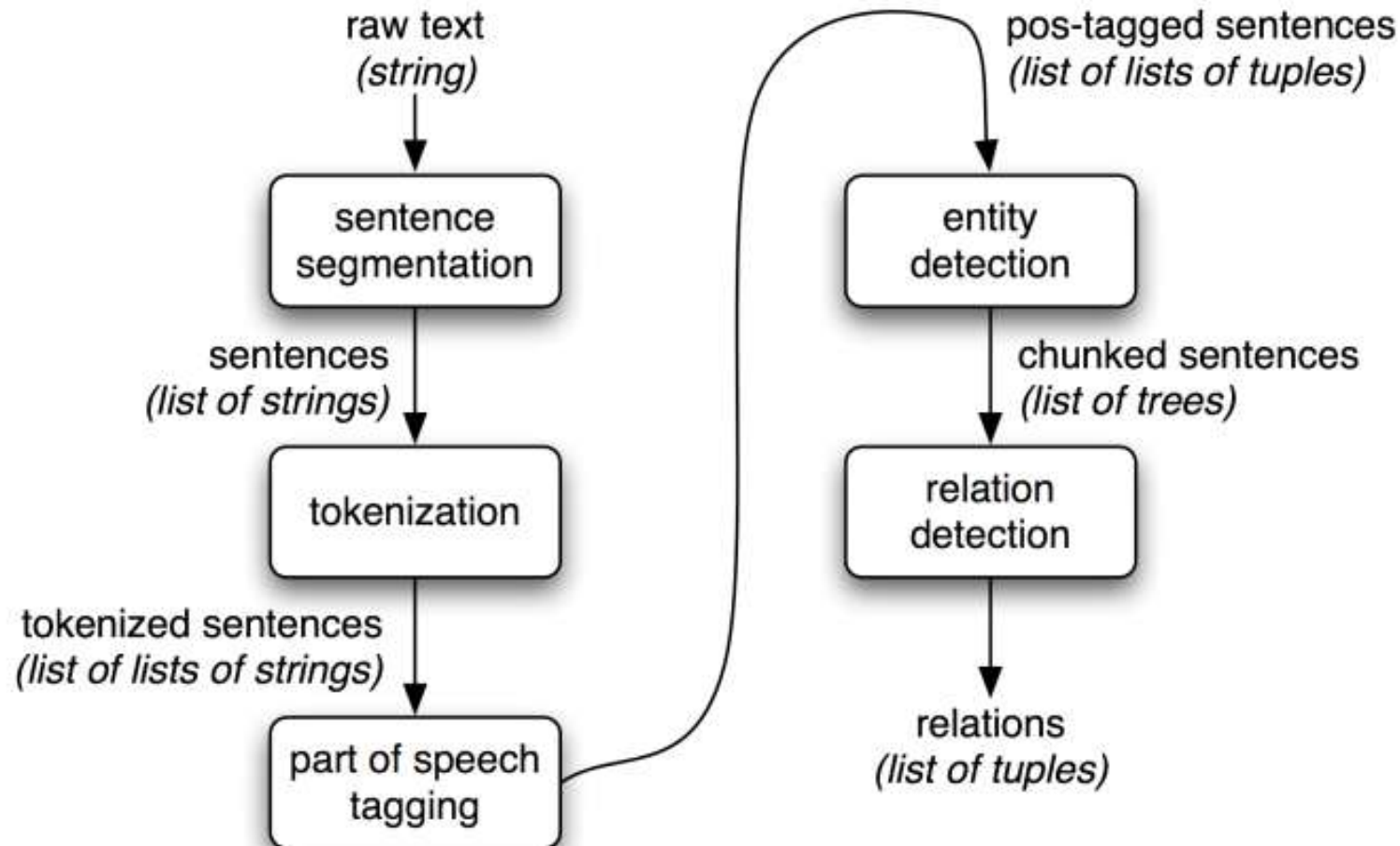
Step 6

Shallow Parsing

❖ Shallow parsing (also chunking or light parsing) is an analysis of a sentence which first identifies constituent parts of sentences (nouns, verbs, adjectives, etc.) and then links them to higher order units that have discrete grammatical meanings (noun groups or phrases, verb groups, etc.).



Shallow Parsing



Shallow Parsing

```
import nltk
sentence = [
    ("a", "DT"),
    ("clever", "JJ"),
    ("fox", "NN"),
    ("was", "VBP"),
    ("jumping", "VBP"),
    ("over", "IN"),
    ("the", "DT"),
    ("wall", "NN")
]
grammar = "NP:{<DT>?<JJ>*<NN>}"
Reg_parser = nltk.RegexpParser(grammar)
Reg_parser.parse(sentence)
Output = Reg_parser.parse(sentence)
Output.draw()
```

Dependency-based Parsing

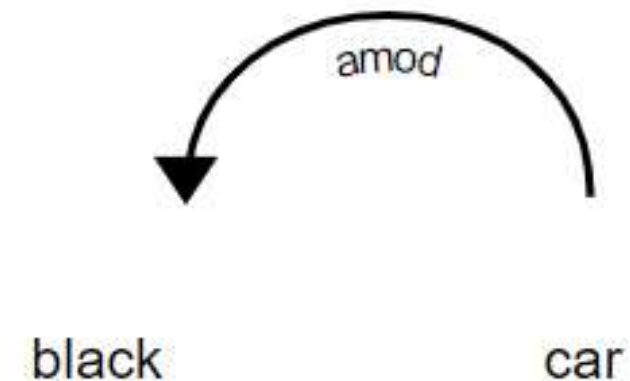
- ❖ **Dependency parsing** displays only relationships between words and their constituents while **constituency parsing** displays the entire sentence structure and relationships.
- ❖ All the nodes in the tree are words; links among words are labeled by syntactic function.
- ❖ A dependency parse tree is a graph where the set of vertices contains the words in the sentence, and each edge in connects two words.
- ❖ The graph must satisfy three conditions:
 - There has to be a single root node with no incoming edges.
 - For each node in , there must be a path from the root to .
 - Each node except the root must have exactly 1 incoming edge.
 - Additionally, each edge in has a type, which defines the grammatical relation that occurs between the two words.

Dependency-based Parsing

Each relationship:

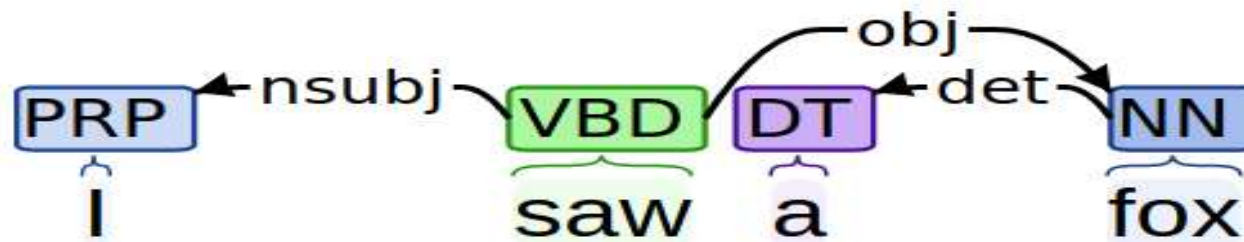
- ❖ Has one **head** and a **dependent** that modifies the **head**.
- ❖ Is labeled according to the nature of the dependency between the **head** and the **dependent**. These labels can be found at [Universal Dependency Relations](#).

A relationship between ***car*** and ***black*** because *black* modifies the meaning of *car*. Here, *car* acts as the **head** and *black* is a **dependent** of the head. The nature of the relationship here is **amod** which stands for “**Adjectival Modifier**”. It is an adjective or an adjective phrase that modifies a noun.



Dependency-based Parsing

Dependency-based Parsing



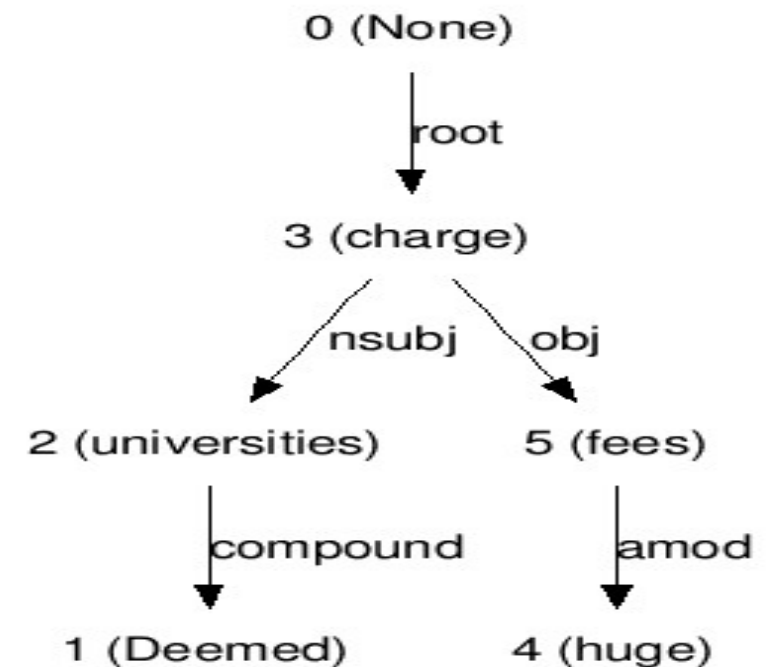
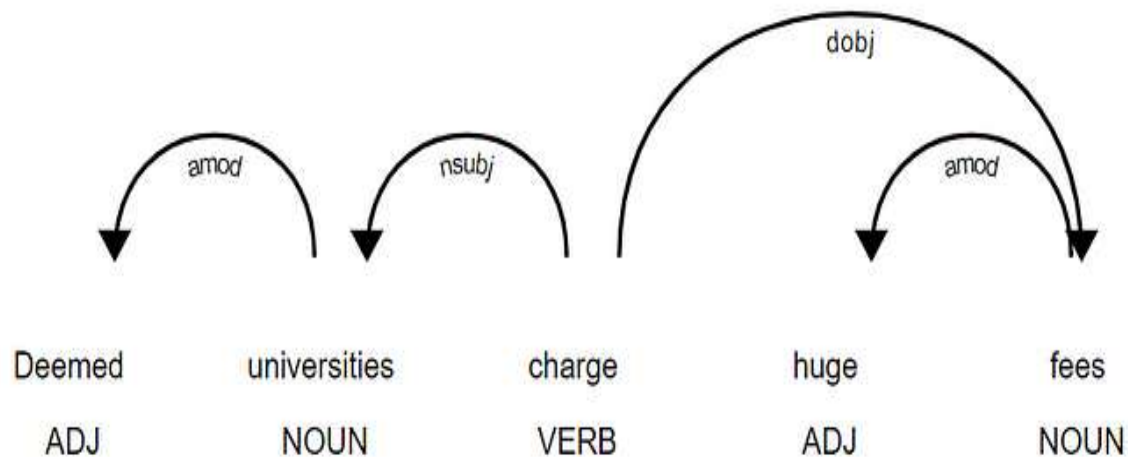
The root of the tree is the **verb** of the sentence, and edges between words describe their relationships.

For example, the word "saw" has an outgoing edge of type *nsubj* to the word "I", meaning that "I" is the nominal subject of the verb "saw".

In this case, we say that "I" *depends on* "saw".

Representation of Syntactic Structure

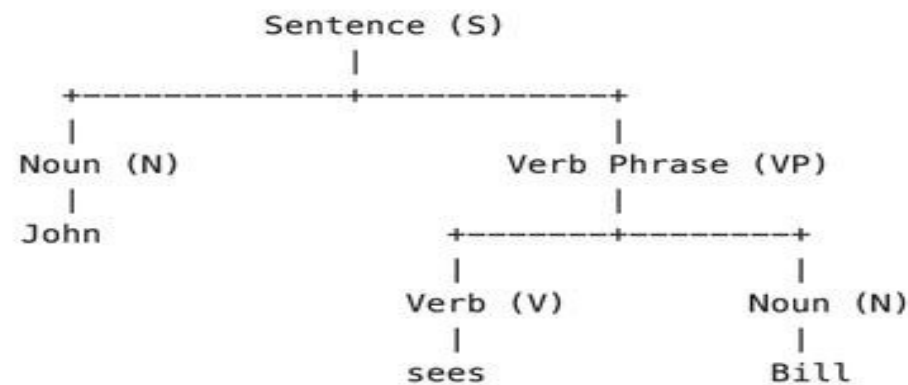
Dependency-based Parsing



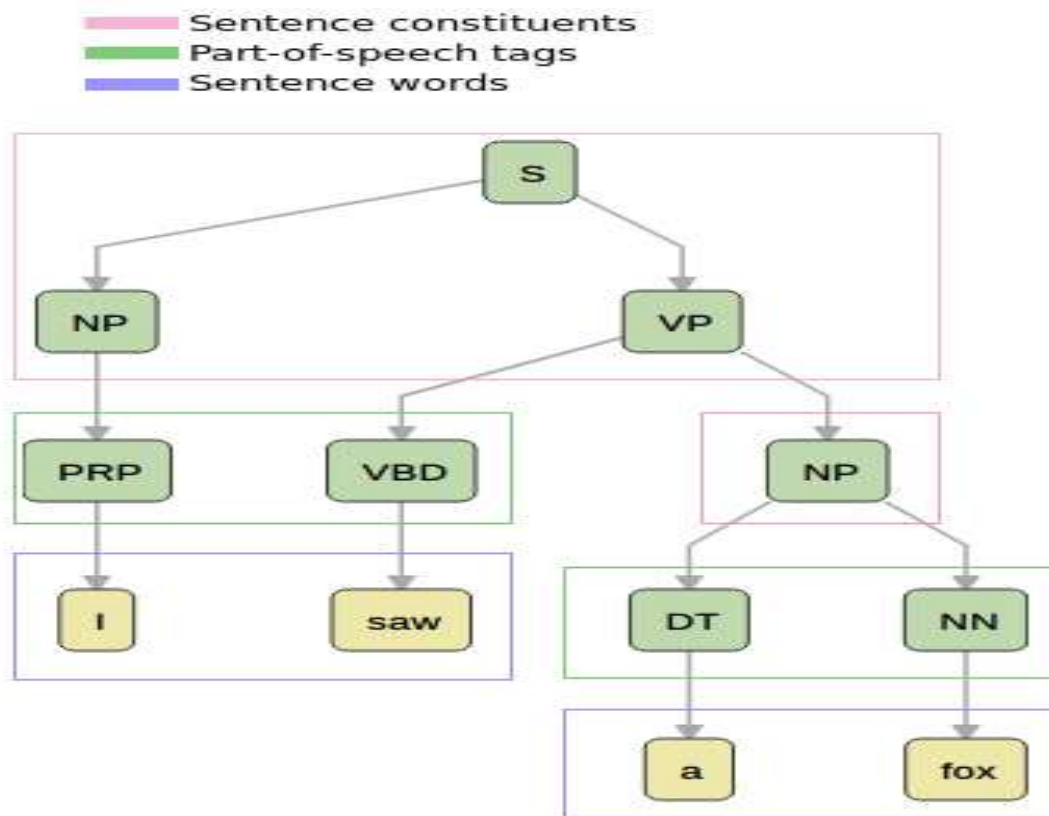
Phrase Structured Parsing

Constituency -based Parsing

- ❖ Constituency Parsing is the process of analysing the sentences by breaking down it into sub-phrases also known as constituents.
- ❖ These sub-phrases belong to a specific category of grammar like NP (noun phrase) and VP(verb phrase).
- ❖ Constituency parsing aims to extract a constituency-based parse tree from a sentence that represents its syntactic structure according to a [phrase structure grammar](#) or CFG



Phrase Structured Parsing

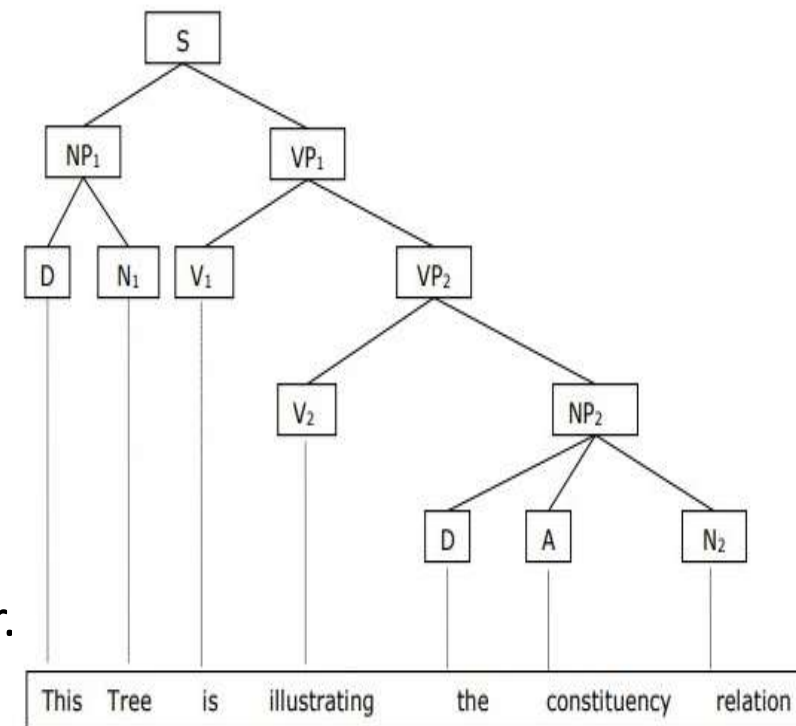


Parsing Natural Language

Constituency Grammar (CG)

- ❖ It is also known as Phrase structure grammar. It is called constituency Grammar as it is based on the constituency relation. It is the opposite of dependency grammar.
- ❖ Before deep dive into the discussion of CG, let's see some fundamental points about constituency grammar and constituency relation.
- ❖ All the related frameworks view the sentence structure in terms of constituency relation.
- ❖ To derive the constituency relation, we take the help of subject-predicate division of Latin as well as Greek grammar.
- ❖ Here we study the clause structure in terms of noun phrase NP and verb phrase VP.

Sentence: This tree is illustrating the constituency relation



Parsing Natural Language

Dependency Grammar (DG)

❖ It is opposite to the constituency grammar and is based on the dependency relation. **Dependency grammar** (DG) is opposite to constituency grammar because it lacks phrasal nodes.

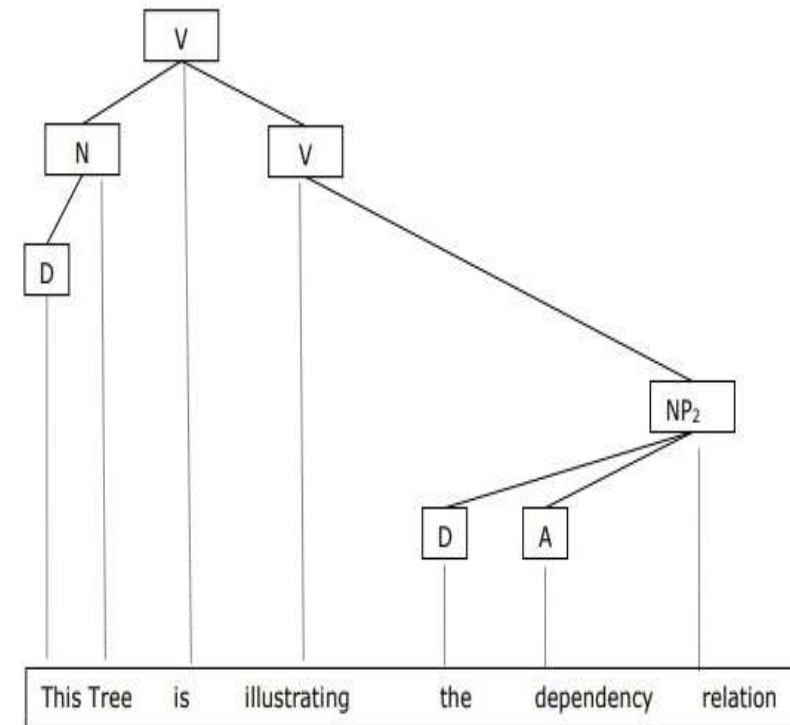
❖ Before deep dive into the discussion of DG, let's see some fundamental points about Dependency grammar and Dependency relation.

❖ In Dependency Grammar, the words are connected to each other by directed links.

❖ The verb is considered the center of the clause structure.

❖ Every other syntactic unit is connected to the verb in terms of directed link. These syntactic units are called dependencies.

Sentence: This tree is illustrating the dependency relation



Parsing Natural Language

Constituency Parsing	Dependency Parsing
Constituency parsing focuses on identifying the constituent structure of a sentence, such as noun phrases and verb phrases.	Dependency parsing focuses on identifying the grammatical relationships between words in a sentence, such as subject-verb relationships.
Constituency parsing uses phrase structure grammar, such as context-free grammar or dependency grammar.	Dependency parsing uses dependency grammar, which represents the relationships between words as labeled directed arcs.
Constituency parsing is based on a top-down approach, where the parse tree is built from the root node down to the leaves.	Dependency parsing is based on a bottom-up approach, where the parse tree is built from the leaves up to the root.
Constituency parsing represents a sentence as a tree structure with non-overlapping constituents.	Dependency parsing represents a sentence as a directed graph, where words are represented as nodes and grammatical relationships are represented as edges.

Parsing Natural Language



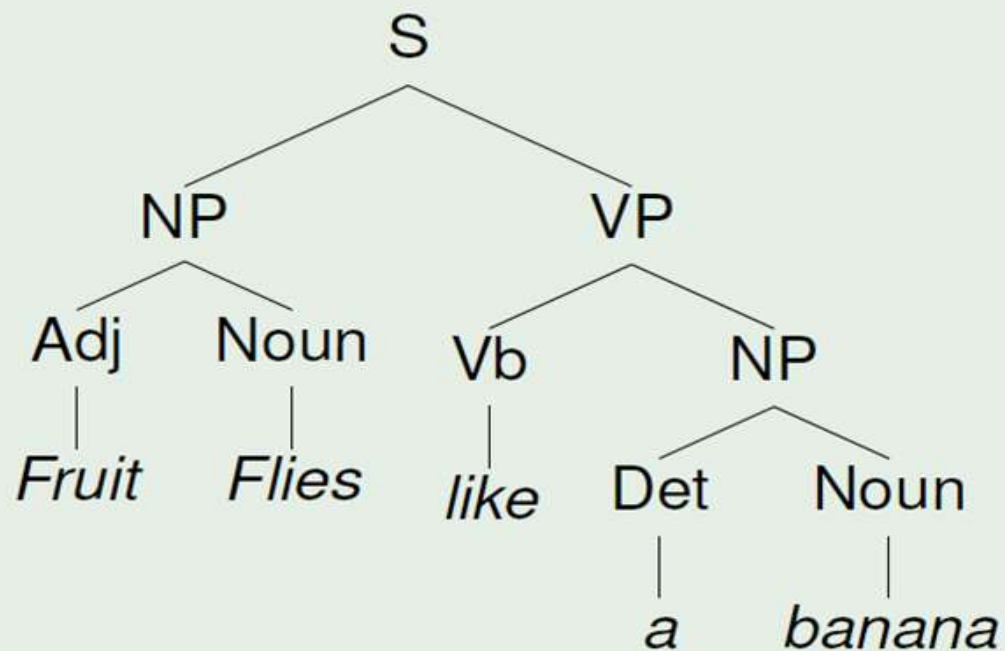
Learning from Experience, Sharing with Passion

Constituency Parsing	Dependency Parsing
Constituency parsing is more suitable for natural language understanding tasks.	Dependency parsing is more suitable for natural language generation tasks and dependency-based machine learning models.
Constituency parsing is more expressive and captures more syntactic information, but can be more complex to compute and interpret.	Dependency parsing is simpler and more efficient, but may not capture as much syntactic information as constituency parsing.
Constituency parsing is more appropriate for languages with rich morphology such as agglutinative languages.	Dependency parsing is more appropriate for languages with less morphological inflection like English and Chinese.
Constituency parsing is more suitable for natural language understanding tasks.	Dependency parsing is more suitable for natural language generation tasks and dependency-based machine learning models.
Constituency parsing is used for more traditional NLP tasks like Named Entity Recognition, Text classification.	Dependency parsing is used for more advanced NLP tasks like Machine Translation, Language Modeling, and

Parsing Natural Language

Fruit flies like a banana

Constituency Structure



Dependency Structure

